The Journal of Open Source Software

Pyrception: A Python package for biologically plausible neuromorphic perception

Alexander Hadjiivanov [1,2] **and Giulia D'Angelo**[3]

1 Advanced Concepts Team, European Space Agency **2** Netherlands eScience Center **3** Czech Technical University in Prague

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Patrick Diehl

Submitted: 09 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Neuromorphic perception is enjoying a resurgence in attention from the scientific community as event-based cameras (ECs) (Lichtsteiner et al., 2008) become increasingly capable and commercially available (Posch et al., 2014). ECs build upon decades of research on retinomorphic vision, display superior characteristics compared to frame-based cameras (FCs) and highlight the advantages of neuromorphic perception. Here, we present `Pyrception` – a Python library for bio-plausible retinomorphic modelling, simulation and processing of raw visual input. The library builds upon a generalisation of sparse convolution using Toeplitz matrices (outlined below), which allows for efficient processing and modelling of various features of the mammalian retina, such as complex receptive fields, eccentricity-dependent receptive field sizes and log-polar cell arrangement (Chessa et al., 2016).

Statement of need

Hardware prototyping is a difficult and time-consuming process. This is especially valid for neuromorphic vision sensors as the technology is new, and investing effort into developing a new sensor implementing innovative on-sensor features may not be justified unless it has already been demonstrated that the processing pipeline works as intended. `Pyrception` can be used to simulate retinal processing circuits, with envisioned applications in the fields of neuromorphic vision and neuroscience. The ability to perform accurate simulations of various retinal circuits is an efficient way to demonstrate any potential advantages of such circuits and subsequently presenting a strong case for their implementation in hardware. Similarly, such simulation can prove beneficial in neuroscience, for instance, for modelling atypical or poorly understood visual mechanisms to gain insight into the causes and mechanisms behind of certain vision-related medical conditions.

Implementation

The implemention of convolution in deep learning libraries such as PyTorch (Paszke et al., 2019) and TensorFlow (Abadi et al., 2016) is based on the `Im2Col` mechanism (**?**). Since in the case of digital visual input the convolution operation is discrete, in the `Im2Col` approach the input (an image) is partitioned into patches corresponding to each convolved location, which is determined by factors such as kernel size, stride, dilation and so forth. Each of these patches is unrolled into a column vector and concatenated to form the columns of a dense matrix, hence the term `Im2Col` (Fig. 1a). The same operation is performed on all kernels – they are unrolled and concatenated as the *rows* of a separate dense matrix (Fig. 1b). This operation reduces the convolution operation to a simple dense matrix-matrix multiplication.

While the `Im2Col` approach makes the convolution operation well suited for implementation on

DRAFT

GPUs, it has several limitations that make it unsuitable for modelling the working principle of the retina. For instance, `Im2Col` requires that all kernels be of the same size, precluding the emulation of log-polar dependent receptive field architecture. Similarly, it makes it exceedingly difficult to implement non-rectangular (e.g., elliptic or irregular) kernels.

In contrast, in `Pyrception` the image is not segmented into patches. Instead, it is unrolled into a single long vector by concatenating all columns of the image (Fig. 1c). As a consequence, the parts of the image that correspond to the receptive field of a kernel are no longer contiguous. To be able to still perform convolution as a simple matrix-matrix (strictly, matrix-vector) multiplication, each corresponding kernel must be unrolled into a *sparse* row in such a way that multiplying the sparse row with the entire image column vector gives the same result as if performed with the `Im2Col` approach (Fig. 1d). This type of convolution is based on Toeplitz matrices (Gnacik & Łapa, 2022), where all diagonals contain identical elements, which is equivalent to shifting consecutive rows by one element to the right. In `Pyrception`, kernels are unravelled into sparse rows, one for each convolved patch of the image, resulting in a sparse Toeplitz matrix.

The versatility of `Pyrception` stems from using a *generalised* version of the sparse convolution approach with a Toeplitz matrix. The 'generalised' part refers to the fact that each patch of the image can be convolved with a *different* kernel since each sparse rows in the kernel matrix encodes an entire kernel. Since each row in the kernel matrix has the same length as the image vector, each kernel can potentially have a full 'view' of the entire image. This gives us complete freedom to choose the parameters of each kernel independently, making it possible to construct convolutional matrices populated with arbitrary kernels with different sizes, orientations and arrangements. Notably, the kernels themselves can be sparse, making it possible to convolve sparse inputs (e.g., inputs arriving from an event camera).

Currently, `Pyrception` implements all five major layers of the mammalian retina, each of which can be emulated with different types of cell arrangements and dynamics, as well as excitatory / inhibitory connections forming receptive fields of different shapes, sizes and orientations. With the default parameters, the `Pyrception` layers implement a generic retinal processing pipeline consisting of a receptor layer followed by horizontal, bipolar, amacrine and ganglion cell layers (Masland, 2011). By simply deriving from the existing layer classes and overriding the `forward()` method, it is possible to implement different retinal circuits, such as motion segmentation (Baccus et al., 2008; Clerico et al., 2024; Ölveczky et al., 2003), looming detection for detecting approaching objects (Gollisch & Meister, 2010), optical flow estimation and visual tracking (Angelo et al., 2025; Javier Traver & Bernardino, 2010). We welcome contributions to `Pyrception` that would make the library as useful as possible, including implementing bio-plausible modalities other than vision.

DRAFT

The Journal of Open Source Software

Figures

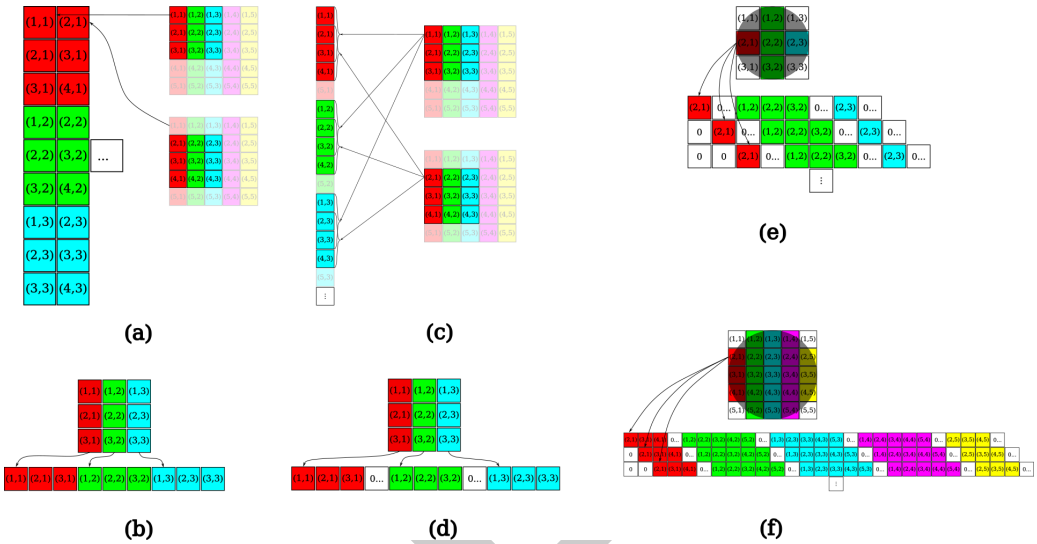


Figure 1: Visualisation of dense and sparse operations on images and kernels for convolution. (a) Unrolling image patches into a dense matrix (`Im2Col` operation) and (b) a corresponding kernel into a dense row (`Ker2Row` operation). (c) Unrolling an image column-wise into a vector (`Im2Vec` operation). The (non-contiguous) elements corresponding to the same patches as those in (a) are illustrated. (d) Unrolling a kernel into a sparse row (`Ker2SpRow` operation). (e, f) Examples of circular kernels with radii of 3 and 5 pixels, respectively, with the corresponding unrolled `Ker2SpRow` representations shown below each kernel.

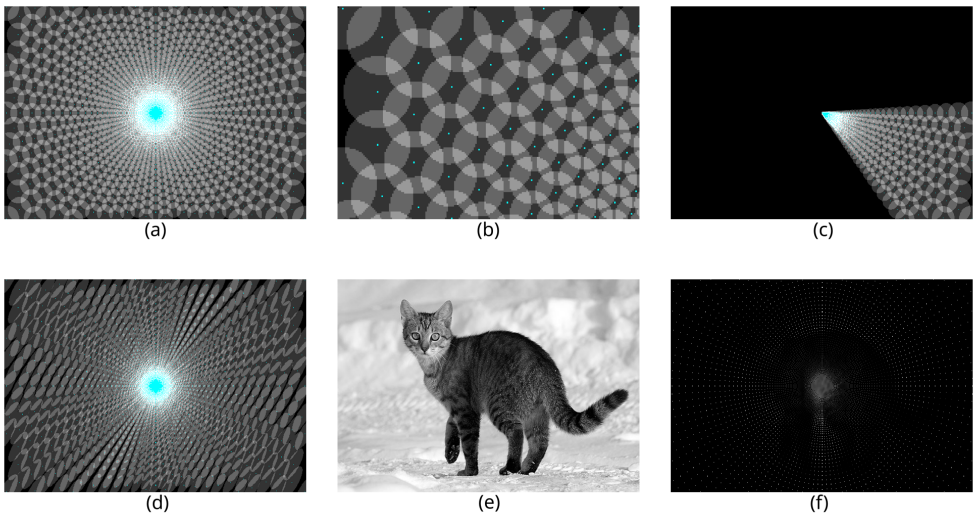


Figure 2: A visualisation of kernels of different sizes and orientations. (a) Kernels arranged and a phyllotactic log-polar distribution (similar to the seeds of a sunflower). Kernels towards the center of the image are smaller than ones towards the edges, emulating the eccentricity-dependent receptive field size observed in the mammalian retina. Due to the different sizes of the kernels, the 'stride' (and therefore the overlap) is non-uniform. (b) A zoomed-in version of the top left corner of the plot in (a). (c) A plot of the kernels for a subset of 10 sectors of kernels (a sector is a group of kernels whose centers are located along the same radial line). (d) The same kernel arrangement as in (a), but with elliptic kernels oriented at 45°. (e) A raw image (credit: Von.grzanka / Wikimedia) and (f) the corresponding activation map resulting from convolving the image with the kernel map in (a).

The Journal of Open Source Software

Acknowledgements

A.H. is grateful for the support of the Advanced Concepts Team of the European Space Agency, where the conceptualisation and a large portion of the development of this library took place. G.D. acknowledges the financial support from the European Union's HORIZON-MSCA-2023-PF-01-01 research and innovation programme under the Marie Skłodowska-Curie grant agreement ENDEAVOR No 101149664.

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2016). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems*. https://doi.org/10.48550/ARXIV.1603.04467

Angelo, G. D., Clerico, V., Bartolozzi, C., Hoffmann, M., Furlong, P. M., & Hadjiivanov, A. (2025). *Wandering around: A bioinspired approach to visual attention through object motion sensitivity*. https://doi.org/10.48550/ARXIV.2502.06747

Baccus, S. A., Ölveczky, B. P., Manu, M., & Meister, M. (2008). A Retinal Circuit That Computes Object Motion. *Journal of Neuroscience*, *28*(27), 6807–6817. https://doi.org/10.1523/JNEUROSCI.4206-07.2008

Chessa, M., Maiello, G., Bex, P. J., & Solari, F. (2016). A space-variant model for motion interpretation across the visual field. *Journal of Vision*, *16*(2), 12. https://doi.org/10.1167/16.2.12

Clerico, V., Snyder, S., Lohia, A., Kaiser, M. A.-A., Schwartz, G., Jaiswal, A., & Parsa, M. (2024). *Retina-Inspired Object Motion Segmentation for Event-Cameras*. Pre-published. https://doi.org/10.48550/ARXIV.2408.09454

Gnacik, M., & Łapa, K. (2022, October 27). *Using Toeplitz Matrices to obtain 2D convolution*. https://doi.org/10.21203/rs.3.rs-2195496/v1

Gollisch, T., & Meister, M. (2010). Eye Smarter than Scientists Believed: Neural Computations in Circuits of the Retina. *Neuron*, *65*(2), 150–164. https://doi.org/10.1016/j.neuron.2009.12.009

Javier Traver, V., & Bernardino, A. (2010). A review of log-polar imaging for visual perception in robotics. *Robotics and Autonomous Systems*, *58*(4), 378–398. https://doi.org/10.1016/j.robot.2009.10.002

Lichtsteiner, P., Posch, C., & Delbruck, T. (2008). A 128\texttimes{128} 120 dB 15 μs Latency Asynchronous Temporal Contrast Vision Sensor. *IEEE Journal of Solid-State Circuits*, *43*(2), 566–576. https://doi.org/10.1109/JSSC.2007.914337

Masland, R. H. (2011). Cell Populations of the Retina: The Proctor Lecture. *Investigative Opthalmology & Visual Science*, *52*(7), 4581–4591. https://doi.org/10.1167/iovs.10-7083

Ölveczky, B. P., Baccus, S. A., & Meister, M. (2003). Segregation of object and background motion in the retina. *Nature*, *423*(6938), 401–408. https://doi.org/10.1038/nature01652

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. https://doi.org/10.48550/ARXIV.1912.01703

Posch, C., Serrano-Gotarredona, T., Linares-Barranco, B., & Delbruck, T. (2014).

DRAFT

Retinomorphic Event-Based Vision Sensors: Bioinspired Cameras With Spiking Output. *Proceedings of the IEEE*, *102*(10), 1470–1484. https://doi.org/10.1109/JPROC.2014.2346153

DRAFT